Computer Architectures
Exam of 03/02/2026

B1

Please read the following notes before proceeding

1) It is recommended to save the project on the C drive during development. Once you have completed your work, you must save the project inside the exam folder on the Z drive (the same folder where you found this document). **Projects saved elsewhere will not be collected and will be permanently lost.**
2) The assembly subroutine developed in the first exercise must comply with the ARM Architecture Procedure Call Standard (AAPCS) standard (in terms of parameter passing, returned value, callee-saved registers).
3) If you prefer, you can begin with the second exercise. Since it requires calling the assembly subroutine from the first exercise, you can create a stub in the assembly file to proceed. For example:

```
subroutineName    PROC
      ; to do: add code
      MOV r0, #10       ; example of a return value
      BX LR
      ENDP
```

4) To earn points on all exercises, it is recommended that you write at least some code for each.
5) **The final project must run on the actual board (e.g., button debouncing is required).**

Special characters with Italian keyboards

- Square brackets [ ]
  - Left bracket: ATL GR + [
  - Right bracket: ALT GR +]
- Curly brackets { }
  - Left bracket: LSHIFT + ALT GR + [
  - Right bracket: LSHIFT + ALT GR +]
- Hash # : ALT GR + à

Bitwise operations in C

Left shift by N bits = A<<N
Right shift by N bits = A>>N

A XOR B = A^B
A AND B = A&B (&& is the logical AND)
A OR B = A|B (|| is the logical OR)

Computer Architectures
Exam of 03/02/2026

B1

Question 1 (10 points)

In the look-and-say sequence, each term is generated by describing the digits of the previous term, grouping consecutive identical digits and counting how many times each digit occurs. For example, the number 3668999 generates 13261839 because:

- 3 is read off as "one 3" → 13
- 66 is read off as "two 6s" → 26
- 8 is read off as "one 8" → 18
- 999 is read off as "three 9s" → 39.

Write the `Look_and_say` subroutine in ARM assembly language. You must follow this prototype:

***uint32_t** Look_and_Say(**uint32_t** digits);*

The subroutine receives an unsigned integer in input, and returns the next element *n* of the look-and-say sequence.
You can assume that:

- each digit occurs at most 9 consecutive times
- no overflow occurs during the computation.
- the next element can be held in a 32 bit unsigned integer

Suggestions. You can extract the digits of a number from least significant to most significant by repeatedly dividing by 10 and taking the remainder. For example:
3668999 / 10 = 366899 with remainder 9
366899 / 10 = 36689 with remainder 9
36689 / 10 = 3668 with remainder 9
3668 / 10 = 366 with remainder 8
366 / 10 = 36 with remainder 6
36 / 10 = 3 with remainder 6
3 / 10 = 0 with remainder 3
You know that there are no more digits when the quotient (result of the division) is 0.

Note that the digits are extracted from right to left; then, they must be processed from left to right in order to compute the next element of the sequence. For each group of consecutive identical digits, first append the count and then the digit by repeated multiplication by 10. In detail, you start by initializing $n = 0$ and in every new iteration $n_{new} = n_{old} * 10 + \text{count}$ and then $n_{new} = n_{old} * 10 + \text{digit}$.
In the example:

- the digit 3 occurs once:
  - $n = 0 * 10 + 1 = 1$
  - $n = 1 * 10 + 3 = 13$
- the digit 6 occurs twice:
  - $n = 13 * 10 + 2 = 132$
  - $n = 132 * 10 + 6 = 1326$
- the digit 8 occurs once:
  - $n = 1326 * 10 + 1 = 13261$
  - $n = 13261 * 10 + 8 = 132618$
- the digit 9 occurs three times:
  - $n = 132618 * 10 + 3 = 1326183$
  - $n = 1326183 * 10 + 9 = 13261839$

Computer Architectures
Exam of 03/02/2026

B1

Question 2 (8 points)

Add the following functionalities to your project.

1) Convert the analog value of the potentiometer to a digital value using the A/D converter. The A/D converter produces a 12-bit digital output. Display the 8 most significant bits of this result using the LEDs. LED 4 corresponds to the most significant bit, and LED 11 corresponds to the least significant bit (of the displayed 8-bit value).

2) When the user presses the *Int0* button, take the last 8 most significant bits produced in the last A/D conversion (i.e., the 8-bit value displayed on the LEDs) and pass this value to the `Look_and_say` subroutine. Then, display the 8 least significant bits of the result on the LEDs. LED 4 corresponds to the most significant bit, and LED 11 corresponds to the least significant bit of the displayed result.